# MANUAL PARA CREAR UN CONECTOR DE IWBASE DE DATOS

*Gaspar Oros - 4°10 – Pablo Nogues*

Este manual explica las dependencias y el procedimiento para crear el conector de base de datos usando la teoría dada por el profesor en classroom.

Enlace al Video: https://youtu.be/5L4krlQpirs

## 1. ¿QUÉ ES UN CONECTOR DE BASE DE DATOS?

Un conector (driver o connector) es una biblioteca de software que funciona como traductor e intermediario entre una aplicación y el motor de base de datos. La aplicación está escrita en JavaScript y trabaja con objetos, mientras que SQLite entiende SQL y se comunica mediante su propio protocolo interno.

El conector realiza estas tareas:

- Recibe una orden de la aplicación, por ejemplo, una instrucción preparada.
- Traduce la orden al protocolo que entiende el motor de base de datos.
- Envía la orden al motor.
- Recoge la respuesta, como filas, cantidad de filas afectadas o errores.
- Convierte la respuesta en objetos y arrays de JavaScript.

## 2. DEPENDENCIAS Y HERRAMIENTAS NECESARIAS

### NODE.JS

Antes de comenzar, debe estar instalado Node.js, que incluye npm. Node.js permite ejecutar el código JavaScript del servidor y npm es el gestor de paquetes que descarga bibliotecas.

Para comprobar la instalación, abre una terminal y ejecuta:

```
node --version
```

```
npm --version
```

### VISUAL STUDIO CODE

Para mi proyecto use Visual Studio Code, lo que me facilito en ciertas partes del proceso como por ejemplo crear el directorio

## 3. CREAR E INICIALIZAR EL PROYECTO

En la terminal de VScode se ejecutan los siguientes comandos

```
npm init -y
```

Qué hace el comando: genera package.json con valores predeterminados. La opción -y acepta automáticamente las preguntas.

## 4. ¿PARA QUÉ SIRVE PACKAGE.JSON?

Es la ficha de identidad del proyecto. Incluye datos como nombre, versión y punto de entrada, además de la lista de dependencias que necesita el proyecto. Permite que otra persona instale las bibliotecas del proyecto ejecutando npm install.

En la sección scripts, el documento propone agregar:

```
"scripts": {\n    "start": "node server.js"\n}
```

Con este script se puede iniciar el servidor escribiendo npm start, en lugar de node server.js.

## 5. INSTALAR LAS DEPENDENCIAS

Desde la carpeta mi-proyecto, ejecuta:

```
npm install express better-sqlite3 cors
```

npm descarga las tres bibliotecas Express, Better-SQLite3 y Cors, y sus dependencias internas en node_modules. También crea o actualiza package-lock.json, que fija las versiones exactas instaladas, y agrega las dependencias a package.json.

## 6. ESTRUCTURA DEL PROYECTO

```
mi-proyecto/
├── public/
│   └── index.html
├── db.js
├── server.js
├── package.json
├── node_modules/
└── datos.db
```

public/ contiene los archivos que ve el navegador, como index.html. db.js configura la base de datos, server.js contiene el servidor Express, package.json describe el proyecto, node_modules guarda las bibliotecas instaladas y datos.db es el archivo de base de datos que se crea automáticamente.

Regla importante del documento: todo lo que está dentro de public/ es público. Por eso, el código del servidor y el archivo .db deben quedar fuera de esa carpeta.

## 7. CREAR EL CONECTOR: ARCHIVO DB.JS

Crea db.js en la raíz del proyecto, al mismo nivel que server.js y fuera de public/. Este archivo centraliza la configuración y la conexión con SQLite.

```
// 1. Importamos el conector
const Database = require('better-sqlite3');
const path = require('path');

// 2. Definimos dónde vivirá el archivo de la base de datos
const rutaArchivo = path.join(__dirname, 'datos.db');

// 3. Abrimos (o creamos) la base de datos
const db = new Database(rutaArchivo);
```

```
// 4. Configuración de rendimiento
db.pragma('journal_mode = WAL');

// 5. Creamos la tabla si no existe
db.exec(`
  CREATE TABLE IF NOT EXISTS usuarios (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    password TEXT NOT NULL,
    fecha_registro TEXT DEFAULT CURRENT_TIMESTAMP
  )
`);

// 6. Exportamos la conexión para usarla en otros archivos
module.exports = db;
```

## 8. EXPLICACIÓN DE LAS PARTES DEL CONECTOR

### 1. IMPORTAR BETTER-SQLITE3

const Database = require('better-sqlite3'); importa el paquete instalado. require() carga el módulo y Database se utiliza como constructor para abrir la base de datos.

### 2. IMPORTAR PATH

const path = require('path'); carga un módulo incluido en Node.js. path.join() construye rutas de archivos de forma adecuada para distintos sistemas operativos.

### 3. DEFINIR LA RUTA DEL ARCHIVO

path.join(__dirname, 'datos.db') une la ruta absoluta de la carpeta donde está db.js con el nombre datos.db. Así, la base de datos queda en una ubicación fija aunque el comando se ejecute desde otra carpeta.

### 4. ABRIR O CREAR LA BASE DE DATOS

new Database(rutaArchivo) crea el objeto de conexión y abre el archivo. Si datos.db no existe, better-sqlite3 lo crea automáticamente; si existe, lo abre. La variable db representa la conexión usada para trabajar con la base.

### 5. CONFIGURAR WAL

db.pragma('journal_mode = WAL') ajusta el modo de registro de SQLite. WAL significa Write-Ahead Logging. Según el documento, permite que varios procesos lean mientras uno escribe y puede ayudar a evitar errores de base de datos bloqueada. La guía indica que es opcional.

### 6. CREAR LA TABLA

db.exec() ejecuta instrucciones SQL que no devuelven resultados, como crear tablas. CREATE TABLE IF NOT EXISTS usuarios crea la tabla solamente si todavía no existe; por eso se puede ejecutar el archivo varias veces sin volver a crear una tabla ya existente.

### 7. EXPORTAR LA CONEXIÓN

module.exports = db; permite que otros archivos usen el objeto de conexión. En server.js se puede escribir const db = require('./db');. De esta forma, db.js se ocupa de la base de datos y server.js del manejo de peticiones HTTP.

## 9. COLUMNAS DE LA TABLA USUARIOS

- **id:** INTEGER PRIMARY KEY AUTOINCREMENT: identificador entero único. SQLite asigna automáticamente el siguiente número.
- **nombre:** TEXT NOT NULL: texto obligatorio.
- **email:** TEXT NOT NULL UNIQUE: texto obligatorio y sin valores repetidos.
- **password:** TEXT NOT NULL: campo obligatorio para la contraseña. El documento señala que más adelante se verá que nunca debe guardarse en texto plano.
- **fecha_registro:** TEXT DEFAULT CURRENT_TIMESTAMP: si no se envía un valor, SQLite guarda automáticamente la fecha y hora actual.

## 10. PROBAR EL CONECTOR

Abre una terminal en la carpeta del proyecto y ejecuta:

```
node db.js
```

No aparecerá ningún mensaje porque el archivo no imprime nada. Sin embargo, al revisar la carpeta del proyecto debería aparecer datos.db, que ya contiene la tabla usuarios. Sin embargo si se sigue la guía de cómo hacer el Server.js debería aparecer: `Servidor escuchando en http://localhost:3000`